MAL • DATA PLATFORM • PART 3 OF 3

# Real-Time Streaming Architecture

A narrow, minutes-scale path for two fraud features the 24-hour batch CAR cannot serve fast enough.

**PART 3 OF 3 — REAL-TIME STREAMING ARCHITECTURE**

Prepared for: Engineering, AI/ML, Product • Status: Draft for review • Companion: Deliverable 1 (Architecture), Part 2 (Trade-offs)

Two fraud signals move from the 24-hour batch CAR to a minutes-scale streaming path: `txn_velocity_5min` (a trailing five-minute transaction count per `ecid`) and `new_device_flag` (an unrecognized device attempting a transaction). Both lose almost all their value if they are a day stale. No other feature is in scope – this does not reopen batch as the right choice for the rest of the CAR (Part 2, Section 1.5). Every event entering the pipeline already carries an `ecid` resolved by Deliverable 1's identity waterfall; the streaming layer never performs identity resolution of its own.

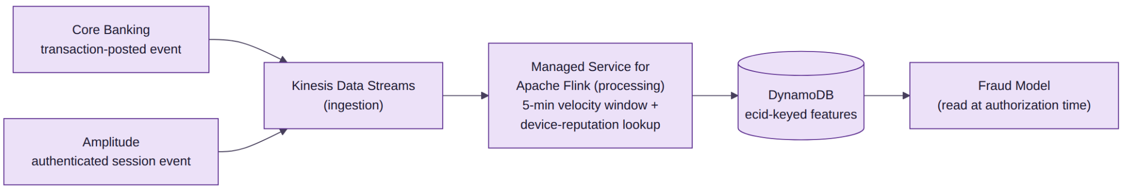


*Figure 1.* Both event sources already exist upstream; only ingestion, processing, and the serving table are new. Computed features are read directly by the fraud model at authorization time – never merged into `CAR_SNAPSHOT`.

## Technology Choice

Kinesis Data Streams paired with Amazon Managed Service for Apache Flink was selected over two alternatives: Amazon MSK Serverless + Flink (the same no-capacity-planning profile on the open Kafka API – kept as the credible fallback if a future integration needs Kafka-native consumers) and a hand-rolled DynamoDB Streams + Lambda approach (lowest effort, but the wrong tool once real load arrives, since it approximates a stateful windowed aggregation with per-write invocations instead of using one). Kinesis + Flink wins for the same reason FindMatches won in Deliverable 1: fully managed on both ends, with windowing that is a genuine capability match for a trailing five-minute count. This does add a second managed-streaming dependency to an already AWS-concentrated stack – the same vendor-concentration trade-off Part 2 (Section 4) already accepts, not a new one.

## Boundaries and Consistency

Everything else in `CAR_SNAPSHOT` stays on the nightly batch path exactly as Part 2 (Section 1.5) argued; this design adds one bounded exception, not a reopening of that decision. The streaming features are not new `CAR_SNAPSHOT` columns – they live in a separate DynamoDB table the fraud model joins by `ecid` at inference time, so Deliverable 2's ERD does not need to change. Consent treatment follows precedent: both features are computed under the same legitimate-interest basis already applied to the CAR's Transaction Behavior group, so neither is gated by personalization consent.

## Open Risks

- PII tier wiring for the new DynamoDB table is not yet specified; it should inherit Tier 1 tokenized-pseudonym access.
- The five-minute freshness target is a design commitment, not a load-tested number.
- A bad identity merge leaves stale entries under the old `ecid` until the next event re-keys it – the same exposure Part 2 (Section 1.6) already named for the batch path, not a new one.


Adithya Menon • amenon3002@gmail.com